

Uniswap v2 核心合约

海登·亚当斯
hayden@uniswap.org

诺亚·兰格
noah@uniswap.org

丹·瑞安
dan@paradigm.xyz

2020 年 3 月

摘要

本技术白皮书阐释了 Uniswap v2 核心合约背后的部分设计决策。白皮书涵盖了合约的新功能——包括 ERC20 代币间的任意交易对、经过强化的价格预言机（该预言机允许其他合约估算给定时间区间内的时间加权平均价格）、“闪电兑换”（允许交易者先接收资产并在交易后续环节完成支付前将其用于其他用途），以及未来可启用的协议手续费。白皮书还对合约进行了架构重构，以缩小其攻击面。本白皮书详细介绍了 Uniswap v2 “核心”合约的运行机制，包括存储流动性提供商资金的交易对合约，以及用于实例化交易对合约的工厂合约。

1 引言

Uniswap v1 是以太坊区块链上一套基于智能合约的链上系统，它依据“恒定产品公式”实现了自动化流动性协议 [1]。每个 Uniswap v1 交易对都存储两种资产的资金池储备，并为这两种资产提供流动性，同时维持储备乘积不会减少这一不变性。交易者在交易时需支付 30 个基点的手续费，该费用归流动性提供者所有。这些合约不具备可升级性。

Uniswap v2 是基于同一公式的全新实现版本，具备多项极具价值的新功能。最关键的是，它支持创建任意 ERC20/ERC20 交易对，而非仅支持 ERC20 与 ETH 之间的交易对。同时，它还提供了经过强化的价格预言机，会在每个区块起始时记录两种资产的相对价格。这使得以太坊上的其他智能合约能够估算任意时间段内两种资产的时间加权平均价格。最后，该版本实现了“闪电兑换”功能，用户可免费获取资产并在链上其他地方使用，仅需在交易结束时支付（或归还）这些资产。

虽然该合约通常不可升级，但存在一个私钥，该私钥可更新工厂合约中的一个变量，从而开启链上 0.05% 的交易手续费。该手续费最初将处于关闭状态，但未来可能开启；开启后，流动性提供者每笔交易将获得 0.25% 的收益，而非原本的 0.30%。

正如 3.1 节所述，Uniswap v2 还修复了 Uniswap v1 中的一些小问题，同时对实现架构进行了重新设计：通过最小化存放流动性提供者资金的“核心”合约中的逻辑，减少了 Uniswap 的攻击面，并让系统更易于升级。

本文阐述了该核心合约的运行机制，以及用于实例化这些合约的工厂合约。实际使用 Uniswap v2 时，需通过“路由器”合约调用交易对合约，由该路由器计算交易或存款金额并将资金转至交易对合约。

2 新功能

2.1 ERC-20 交易对

Uniswap v1 使用 ETH 作为桥接货币。每个交易对都将 ETH 作为其资产之一。这使得路由更简单——ABC 与 XYZ 之间的每笔交易都要通过 ETH/ABC 交易对和 ETH/XYZ 交易对进行——并减少了流动性的碎片化。

然而，这一规则给流动性提供者带来了显著成本。所有流动性提供者都持有 ETH 敞口，并会根据其他资产相对于 ETH 的价格变动承受无常损失。当 ABC 和 XYZ 两种资产存在相关性时——例如二者均为美元稳定币——Uniswap 交易对 ABC/XYZ 上的流动性提供者通常承受的无常损失，会低于 ABC/ETH 或 XYZ/ETH 交易对的流动性提供者。

将 ETH 用作强制桥接货币也会给交易者带来成本。交易者需要支付的手续费是直接交易 ABC/XYZ 交易对时的两倍，并且会遭遇两次滑点。

Uniswap v2 允许流动性提供者对任意两种 ERC-20 代币创建交易对合约。

任意 ERC-20 代币对的数量激增可能会让找到交易某一特定代币对的最佳路径变得稍显困难，但路由机制可在更高层级进行处理（无论是在链下实现，还是通过链上路由聚合器来完成）。

2.2 价格预言机

Uniswap 在时刻 t 提供的边际价格（不含手续费）可通过将资产 a 的储备金除以资产 b 的储备金计算得出

$$p_t = \frac{r_a}{r_b} (1)$$

由于如果该价格存在偏差（偏差幅度足以覆盖手续费），套利者会与 Uniswap 进行交易，因此 Uniswap 提供的价格往往会追踪资产的相对市场价格，这一点正如 Angeris 等人在文献 [2] 中所展示的。这意味着 Uniswap 可被用作近似价格预言机。

然而，Uniswap v1 并不适合作为链上价格预言机，因为它极易被操纵。假设其他某个合约依据当前 ETH-DAI 价格来结算衍生品，想要操纵该价格的攻击者可以从 ETH-DAI 交易对买入 ETH，触发衍生品合约的结算（使其基于……进行结算）

以被抬高的价格完成结算，随后再将以太币卖回交易对，使价格回落至真实水平。

这甚至可以作为一笔原子交易来完成，也可以由控制区块链内交易顺序的矿工来执行。

Uniswap v2 通过测量并记录每个区块第一笔交易前（或等价地，前一区块最后一笔交易后）的价格，改进了这一预言机功能。

1 关于将 Uniswap v1 用作预言机会使合约易受此类攻击的真实案例，参见 [3]。

区块)。这种价格比单个区块内的价格更难被操纵。如果攻击者提交一笔试图在区块末尾操纵价格的交易，其他一些套利者或许能在同一区块内紧随其后提交另一笔反向交易。矿工（或拥有足够燃气填满整个区块的攻击者）或许能在区块末尾操纵价格，但除非他们同时挖出下一个区块，否则在反向交易的套利操作上未必具备特殊优势。

具体而言，Uniswap v2 会通过记录每次有人与合约交互的区块起始价格的累计总和来累积该价格。根据区块时间戳，每个价格都会按自上一个更新区块以来经过的时间进行加权²。这意味着，在任意给定时间（更新后），累积器的数值应等于合约历史中每一秒的现货价格之和。

$$a_t = \sum_{i=1}^t p_i(2)$$

要估算从时间 t_1 到 t_2 的时间加权平均价格，外部调用方可以先在 t_1 时刻对累加器的值进行检查点记录，随后在 t_2 时刻再次记录，用第二个值减去第一个值，再除以经过的秒数。（需注意，合约本身不会存储该累加器的历史值——调用方必须在周期开始时调用合约，读取并存储该值。）

$$p_{t_1:t_2} = \frac{\sum_{i=t_1}^{t_2} p_i}{t_2 - t_1} = \frac{\sum_{i=1}^{t_2} p_i - \sum_{i=1}^{t_1} p_i}{t_2 - t_1} = \frac{a_{t_2} - a_{t_1}}{t_2 - t_1} (3)$$

预言机的用户可以选择该时间段的开始和结束时间。选择更长的时间段会让攻击者操纵 TWAP 的成本更高，不过这会导致价格的时效性变差。

一个复杂之处在于：我们应该以资产 B 为单位衡量资产 A 的价格，还是以资产 A 为单位衡量资产 B 的价格？尽管以 B 计价的 A 的现货价格始终是 A 计价的 B 的现货价格的倒数，但在特定时间段内，以 B 计价的资产 A 的平均价格并不等于以 A 计价的资产 B 的平均价格的倒数。³ 例如，如果区块 1 中美元对以太坊的价格为 100，区块 2 中为 300，那么美元对以太坊的平均价格为 200 美元 / 以太坊，而以太坊对美元的平均价格则为 1/150 以太坊 / 美元。由于合约无法确定用户希望将哪一种资产作为记账单位，Uniswap v2 会对两种价格都进行追踪。

另一个复杂情况是，用户有可能在不与交易对合约交互的情况下向其发送资产——从而改变合约的余额和边际价格，同时也不会触发预言机更新。如果该合约仅检查自身余额并根据当前价格更新预言机，攻击者便可可在一个区块中首次调用合约前，立即向合约发送一笔资产来操纵预言机。若上一次交易所在区块的时间戳距今为 X 秒，那么即便无人进行相关操作，合约仍会在累加新价格前，错误地将其乘以 X。

² 由于矿工在设置区块时间戳时有一定的自由度，预言机的用户应注意这些数值可能与现实世界的时间并不完全对应。

³ 在特定时期内，资产 A 相对于资产 B 的算术平均价格，等于该时期内资产 B 相对于资产 A 的调和平均价格的倒数。如果合约计量的是几何平均价格，那么两种价格互为倒数。不过，几何平均时间加权平均价格（TWAP）的使用并不普遍，且在以太坊上难以计算。

有机会以该价格进行交易。为防止这种情况发生，核心合约会在每次交互后缓存其储备金，并使用从缓存储备金得出的价格而非当前储备金来更新预言机。除了保护预言机免受操纵外，这一更改还支持下文 3.2 节中所述的合约架构重构。

2.2.1 精度

由于 Solidity 不原生支持非整数数值数据类型，Uniswap v2 采用一种简单的二进制定点格式来编码和处理价格。具体而言，特定时刻的价格以 UQ112.112 格式的数字存储，这意味着小数点两侧各保留 112 位小数精度，且无符号位。这类数字的取值范围为 $[0, 2^{112} - 1]$ ⁴，精度为 2112。

选择 UQ112.112 格式是出于一个务实的原因——由于这些数字可存储在 uint224 中，256 位存储槽会剩余 32 位可用空间。恰好，每个以 uint112 存储的储备量在（压缩后的）256 位存储槽中也会剩余 32 位可用空间。这些空余空间用于上述的累积过程。具体而言，储备量会与至少有一笔交易的最新区块的时间戳一同存储，该时间戳会经过 2^{32} 取模，以适配 32 位空间。此外，尽管任意时刻的价格（以 UQ112.112 数字存储）可确保适配 224 位，但该价格在一个时间区间内的累积值却无法适配。

A/B 和 B/A 累积价格的存储槽末尾额外的 32 位，用于存储价格多次累加产生的溢出位。这一设计意味着，价格预言机仅会为每个区块的首笔交易额外增加三次 SSTORE 操作（当前成本约为 15000 燃气）。

主要的缺点在于，32 位不足以存储那些理论上永远不会溢出的时间戳值。事实上，Unix 时间戳溢出 uint32 类型的日期是 2106 年 2 月 7 日。为了确保该系统在该日期及之后每经过 $2^{32} - 1$ 秒的时间间隔后仍能正常运行，预言机只需在每个时间间隔内至少查询一次价格（该时间间隔约为 136 年）。这是因为核心的累积（以及时间戳取模）方法实际上是支持溢出安全的，也就是说，只要预言机使用正确的（简单）溢出算法计算差值，跨溢出间隔的交易就能得到合理的核算。

2.3 闪电交换

在 Uniswap v1 中，用户用 XYZ 购买 ABC 时，必须先将 XYZ 发送至智能合约，才能收到 ABC。如果用户需要用自己购买的 ABC 来获取支付所用的 XYZ，这种方式就会很不便。例如，用户可能会用这笔 ABC 在其他合约中购买 XYZ，以通过 Uniswap 的价格差异进行套利；或者他们可能会在 Maker 或 Compound 上平仓，通过出售抵押品来偿还 Uniswap 的债务。

Uniswap v2 新增了一项功能，允许用户在付款前先接收并使用某一资产，前提是他们需在同一笔原子交易内完成付款。交换函数会在转出用户请求的代币与执行不变量验证之间，调用一个由用户指定的可选回调合约。回调执行完毕后，合约会检查新的余额并确认不变量得到满足。

⁴ $2^{112} - (2^{112} - 1)$ 这个理论上限在本场景中并不适用，因为 Uniswap 中的 UQ112.112 数值始终由两个 uint112 数值的比值生成。此类比值的最大值为 $2^{112} - 1 = 2^{112} - 1$ 。

(已就已支付金额的费用进行调整后)。如果合同没有足够资金, 系统会撤销整笔交易。

用户也可以使用同一代币偿还 Uniswap 资金池, 而非完成兑换。这实际上等同于允许任何人闪电存入存储在其中的任何资产

一个 Uniswap 资金池 (费用与 Uniswap 交易收取的 0.30% 相同)。⁵

2.4 协议费用

Uniswap v2 包含一个 0.05% 的协议费用, 该费用可开启或关闭。若开启, 这笔费用将被发送至工厂合约中指定的 `feeTo` 地址。

最初, `feeTo` 未设置, 也不收取任何费用。一个预先指定的地址 —— `feeToSetter` —— 可以调用 Uniswap v2 工厂合约的 `setFeeTo` 函数, 将 `feeTo` 设置为其他值。`feeToSetter` 还可以调用 `setFeeToSetter` 来更改 `feeToSetter` 地址本身。

若已设置 `feeTo` 地址, 协议将开始收取 5 个基点的费用。该费用被视为流动性提供者所赚取的 30 个基点费用中的 $\frac{1}{6}$ 部分。也就是说, 交易者将

所有交易继续支付 0.30% 的手续费; 其中 83.3% (即交易金额的 0.25%) 将分配给流动性提供者, 16.6% (即交易金额的 0.05%) 将分配给 `feeTo` 地址。

在交易时收取这 0.05% 的费用会给每笔交易增加额外的燃气成本。为避免这一情况, 累计费用仅在存入或提取流动性时收取。合约会计算累计费用, 并在任何代币铸造或销毁之前, 立即向费用受益人铸造新的流动性通证。 $\sqrt{x \cdot y}$

可通过测量 $\sqrt{k}$ 的增长来计算总收取费用 (即自上次收取费用以来的 $\sqrt{x \cdot y}$)。⁶ 该公式将 t_1 与 t_2 之间的累计费用表示为 t_2 池中流动性的百分比:

$$f_{1,2} = 1 - \frac{\sqrt{k_1}}{\sqrt{k_2}} \quad (4)$$

如果该费用在 t_1 之前已激活, 则 `feeTo` 地址应收取在 t_1 与 t_2 之间累积的 $\frac{1}{6}$ 部分的费用。因此, 我们需要向 `feeTo` 地址铸造新的流动性通证, 以代表该池的 `edc8081b-feb2-477e-9a90-98cf10fb3de` 份额。其中 ϕ 为 $\frac{1}{6}$ 。

也就是说, 我们需要选择 s_m 以满足以下关系, 其中 s_1 是时间 t_1 时的流通股总数量……

$$\frac{s_m}{s_1} = \phi \cdot f_{1,2} \quad (5)$$

经过一些处理, 包括将 $1 - \frac{\sqrt{k_1}}{\sqrt{k_2}}$ 替换为 $f_{1,2}$ 并求解 s_m , 我们

可以将其改写为:

$$s_m = \frac{\sqrt{k_2} - \sqrt{k_1}}{\sqrt{k_1}} \cdot s_1 \quad (6)$$

将 ϕ 设置为 $\frac{1}{6}$, 我们可得到如下公式:

⁵ 由于 Uniswap 对输入金额收取手续费, 因此相对于提取金额的手续费实际上略高: $\frac{1}{1 - \frac{1}{60000}} - 1 = \frac{1}{60000} \approx 0.3009203\%$

我们可以利用这一不变量, 它并未计入新铸造或销毁的流动性代币, 因为我们清楚每次存入或提取流动性时都会收取手续费。

$$s_m = \frac{\sqrt{k_2} - \sqrt{k_1}}{\sqrt{k_2} + \sqrt{k_1}} \cdot s_1$$

假设最初的存款人向一个交易对存入 100 DAI 和 1 ETH，获得 10 份份额。一段时间后（在没有其他存款人参与该交易对的情况下），他们尝试提取资产，此时交易对中持有 96 DAI 和 1.5 ETH。将这些值代入上述公式，我们得到如下结果：

$$s_m = \frac{\sqrt{1.5 \cdot 96} - \sqrt{1 \cdot 100}}{\sqrt{1.5 \cdot 96} + \sqrt{1 \cdot 100}} \cdot 10 \approx 0.0286(8)$$

2.5 池份额的元交易

Uniswap v2 交易对铸造的流动性池份额原生支持元交易。这意味着用户可以通过签名⁷ 授权其流动性池份额的转移，而非通过自身地址发起链上交易。任何人都可以代表用户调用许可函数提交该签名，支付燃气费，还可在同一笔交易中执行其他操作。

3 其他变更

3.1 编程语言 Solidity

Uniswap v1 采用 Vyper 实现，这是一种类 Python 的智能合约语言。Uniswap v2 则使用应用更广泛的 Solidity 开发，因为在其开发阶段，Vyper 尚不支持一些所需功能（例如解析非标准 ERC-20 代币的返回值，以及通过内联汇编访问 chainid 等新操作码）。

3.2 合约重构

Uniswap v2 的一个设计重点是尽可能减小核心交易对合约的规模并降低其复杂性——该合约用于存储流动性提供者的资产。此合约中的任何漏洞都可能造成灾难性后果，因为价值数百万美元的流动性可能被盗取或冻结。

在评估该核心合约的安全性时，最关键的问题是它是否能保护流动性提供商的资产不被盗取或锁定。除了允许资金池中的一种资产兑换为另一种资产这一基本功能外，任何旨在支持或保护交易者的功能都可在“路由合约”中处理。

事实上，甚至部分交换功能也可以提取到路由器合约中。如上所述，Uniswap v2 会存储每种资产的最新记录余额（以防止对预言机机制的特定恶意利用）。新架构利用这一点进一步简化了 Uniswap v1 合约。

在 Uniswap v2 中，卖家会先将资产发送至核心合约，再调用交换函数。随后，合约会对比资产的上次记录余额与当前余额，以此计算自身收到的资产数量。这意味着核心合约是无感知的

⁷ 已签名的消息符合 EIP-712 标准，这与 CHAI 和 DAI 等代币的元交易所使用的标准相同。

与交易者转移资产的方式相关。它可以不是 `transferFrom` 函数，而是元交易，或是未来任何用于授权 ERC-20 代币转移的机制。

3.2.1 费用调整

Uniswap v1 的交易手续费是在执行恒定乘积不变量之前，将存入合约的金额减少 0.3% 来收取的。该合约隐式执行以下公式：

$$(x_1 - 0.003 \cdot x_{in}) \cdot y_1 \geq x_0 \cdot y_0 \quad (9)$$

借助闪电兑换，Uniswap v2 引入了一种可能性，即 x_{in} 和 y_{in} 这两个值可能均不为零（当用户希望使用同一种资产偿还交易对，而非进行兑换时）。为了在正确收取费用的同时处理此类情况，合约被设计为

执行以下不变量：⁸

$$(x_1 - 0.003 \cdot x_{in}) \cdot (y_1 - 0.003 \cdot y_{in}) \geq x_0 \cdot y_0 \quad (10)$$

为了简化链上计算，我们可以将不等式的两边都乘以 1,000,000：

$$(1000 \cdot x_1 - 3 \cdot x_{in}) \cdot (1000 \cdot y_1 - 3 \cdot y_{in}) \geq 100000 \cdot x_0 \cdot y_0 \quad (11)$$

3.2.2 `sync()` 与 `skim()`

为了防范可能更新交易对合约余额的定制化通证实现，并更优雅地处理总供应量可能超过 2^{112} 的通证，Uniswap v2 设有两个救助函数：`sync()`（同步函数）和 `skim()`（提取函数）。

`sync()` 函数在某个交易对的代币异步缩减余额时，可作为一种恢复机制。在此情况下，交易将无法获得最优汇率，且若没有流动性提供者愿意纠正该情况，该交易对将陷入停滞。`sync()` 函数的作用是将合约储备值设为当前余额，从而在这种情况下实现相对平稳的恢复。

`skim()` 函数充当一种恢复机制，用于应对向交易对发送足够数量的通证导致储备的两个 `uint112` 存储槽溢出的情况，否则可能会引发交易失败。如果交易对当前余额与 $2^{112} - 1$ 之间的差额大于 0，`skim()` 允许用户将该差额提取给调用者。

3.3 处理非标准及特殊代币

ERC-20 标准要求 `transfer()` 和 `transferFrom()` 函数返回一个布尔值，用以表明调用成功或失败 [4]。但部分代币（包括泰达币 Tether (USDT)、币安币 Binance Coin (BNB) 等热门代币）对这两个函数中的一个或两个的实现却没有返回值。Uniswap v1 将这些定义不规范的函数缺失的返回值解读为 `false`，即视为转账未成功，并回滚交易，导致本次转账尝试失败。

⁸ 请注意，使用新架构时， x_{in} 并非由用户提供；而是通过测量回调后合约的余额 x_1 ，再从中减去 $((x_0 - x_{out}))$ 计算得出的。此逻辑无法区分在合约被调用前转入合约的资产与在回调期间转入合约的资产。 y_{in} 的计算方式相同，基于 y_0 、 y_1 以及 y_{out} 进行计算。

Uniswap v2 对非标准实现的处理方式有所不同。具体而言，如果 `transfer()` 调用 9 没有返回值，Uniswap v2 会将其视为成功而非失败。这一变更不应影响任何符合标准的 ERC-20 代币（因为在这些代币中，`transfer()` 始终具有返回值）。

Uniswap v1 还假设对 `transfer()` 和 `transferFrom()` 的调用不会触发对 Uniswap 交易对合约的重入调用。某些 ERC-20 代币（包括支持 ERC-777 “钩子” 的代币 [5]）违反了这一假设。为了完全支持此类代币，Uniswap v2 包含了一个“锁”，可直接防止所有更改公共状态的函数发生重入。如 2.3 节所述，这也能防范闪电兑换中用户指定回调引发的重入问题。

3.4 流动性代币供应量的初始化

当新的流动性提供者将代币存入现有的 Uniswap 交易对时，铸造的流动性代币数量将根据现有代币的数量进行计算：

$$s_{minted} = \frac{x_{deposited}}{x_{startina}} \cdot s_{startina} \quad (12)$$

但如果他们是首位存款人呢？在这种情况下， $x_{startina}$ 为 0，因此该公式不适用。

Uniswap v1 将初始份额供应量设定为等于存入的 ETH 数量（以 wei 为单位）。这是一个还算合理的数值，因为如果初始流动性以正确的价格存入，那么 1 个流动性池份额（与 ETH 一样，是一种 18 位小数的代币）的价值约为 2 个 ETH。

然而，这意味着流动性池份额的价值取决于初始存入流动性的比例，而这一比例相当随意，尤其是无法保证该比例能反映真实价格。此外，Uniswap v2 支持任意交易对，因此许多交易对完全不包含以太坊（ETH）。

相反，Uniswap v2 最初铸造的份额等于存入金额的几何平均值：

$$s_{minted} = \sqrt{x_{deposited} \cdot u_{deposited}} \quad (13)$$

该公式确保了流动性池份额在任何时候的价值，本质上都与最初存入流动性的比例无关。例如，假设 1 个 ABC 的当前价格为 100 个 XYZ。如果初始存入的是 2 个 ABC 和 $\sqrt{200}$ 个 XYZ（比例为 1:100），存款人将获得 $\sqrt{2 \cdot 200} = 20$ 份份额。这些份额目前的价值仍应相当于 2 个 ABC 和 200 个 XYZ。再加上累计费用。

如果初始存款为 2 个 ABC 货币单位和 800 个 XYZ 货币单位（比例为 1:400），存款人本应获得 $\sqrt{2 \cdot 800} = 40$ 份份额。

上述公式确保流动性池份额的价值永远不会低于该池储备金的几何平均值。然而，以下情况是可能的：

9 如上文 3.2 节所述，Uniswap v2 核心合约并未使用 `transferFrom()`。

10 这也降低了舍入误差的可能性，因为股份数量的比特数将约等于储备中资产 x 数量的比特数与储备中资产 y 数量的比特数的平均值：

$$\log_2 \sqrt{x \cdot y} = \frac{\log_2 x + \log_2 y}{2} \quad (14)$$

随着时间推移，流动性池份额会通过累积交易手续费或向流动性池“捐赠”而增长。理论上，这可能导致一种情况：最低数量的流动性池份额（ $1e-18$ 池份额）的价值变得极高，使得小型流动性提供者无法提供任何流动性。

为了缓解这一问题，Uniswap v2 会销毁新铸造的前 $1e-15$ （即 0.000000000000001 ）份池份额（这是池份额最小单位的 1000 倍），并将其发送至零地址，而非铸币者。对于几乎任何交易对来说，这一操作的成本都可以忽略不计¹¹。但这会大幅提高上述攻击的成本。为了将流动性池份额的价值提升至 \$100, the attacker would need to donate \$, 攻击者需要向池内注入 100,000 单位的资产，而这部分资产将被永久锁定为流动性。

3.5 以太坊原生资产包装

与以太坊原生资产 ETH 进行交易的接口，与和 ERC-20 代币交互的标准接口有所不同。因此，以太坊上的许多其他协议不支持 ETH，而是使用规范的“包装以太坊”代币 WETH [6]。

Uniswap v1 是个例外。由于每一个 Uniswap v1 交易对都将 ETH 作为其中一种资产，因此直接处理 ETH 是合理的，这能节省手续费。

由于 Uniswap v2 支持任意 ERC-20 交易对，如今再支持未包装的 ETH 已无意义。增加此类支持会使核心代码库的规模翻倍，还可能导致 ETH 与 WETH 交易对之间的流动性分散¹²。原生 ETH 必须先包装为 WETH，才能在 Uniswap v2 上进行交易。

3.6 确定的交易对地址

与 Uniswap v1 一样，所有 Uniswap v2 交易对合约均由单一工厂合约实例化。在 Uniswap v1 中，这些交易对合约通过 CREATE 操作码创建，这意味着此类合约的地址取决于该交易对的创建顺序。Uniswap v2 采用以太坊新的 CREATE2 操作码 [8] 来生成具有确定性地址的交易对合约。这意味着可以在链下计算出某一交易对的地址（若该交易对存在），而无需查询链上状态。

3.7 最大代币余额

为了高效实现预言机机制，Uniswap v2 仅支持最高为 $2^{112} - 1$ 的储备余额。该数值足以支持总供应量超过 1 千万亿的 18 位小数代币。

如果任一储备金余额确实超过 $2^{112} - 1$ ，任何对 swap 函数的调用都将开始失败（原因是 _update () 函数中的一项检查）。要从这种情况中恢复，任何用户都可以调用 skim () 函数，从流动性池中移除多余资产。

11 从理论上讲，在某些情况下这种销毁可能不可忽视，比如高价值零小数位代币之间的交易对。但无论如何，这类交易对并不适合 Uniswap，因为舍入误差会使得交易无法进行。

12 在撰写本文时，Uniswap v1 上流动性最高的交易对之一是 ETH 与 WETH 的交易对 [7]。

参考文献

[1] 海登·亚当斯 (Hayden Adams) . 2018 年。网址:

<https://hackmd.io/@477a09OrOTCbVR3fd1Ozxg/HI9iLsfTz?tvne=view>.

[2] 吉列尔莫·安赫里斯等人。Uniswap 市场分析。2019 年。arXiv: 1911.03380 [q-fin.TR]。

[3] samczsun. 为娱乐与盈利而获取抵押不足的贷款。2019 年 9 月。网址:

<https://samczsun.com/taking-undercollateralized-loans-for-fun-andfor-profit/>.

[4] 法比安·沃格尔施泰特勒与维塔利克·布特林。2015 年 11 月。网址:

<https://eips.ethereum.org/EIPS/eip-20>。

[5] 霍尔迪·拜利纳、雅克·达夫隆与托马斯·沙巴比。EIP 777: ERC777 通证标准。2017 年 11 月。网址: <https://eips.ethereum.org/EIPS/eip-777>。

[6] 雷达。什么是 WETH? 网址: <https://weth.io/>。

[7] Uniswap.info。包装以太坊 (WETH)。网址: <https://uniswap.info/token/0xc02aaa39b223fe8d0a0e5c4f27e>

[8] 维塔利克·布特林。EIP 1014: 精简版 CREATE2。2018 年 4 月。网址:

<https://eips.ethereum.org/EIPS/eip-1014>。

4 免责声明

本文件仅用于提供一般性信息。其不构成投资建议，也不是购买或出售任何投资产品的推荐或邀约，不得用于评估做出任何投资决策的合理性。本文件不得作为会计、法律或税务建议或投资建议的依据。本文件反映的是作者当前的观点，不代表 Paradigm 及其关联机构的立场，也未必反映 Paradigm、其关联机构或与 Paradigm 相关的个人的观点。本文所载观点可能随时变更，且不会另行更新。